INTERNSHIP REPORT

ISRO URSC 2026

PROJECT TITLE

GNSS Based Orbit Determination and Trajectory Prediction

PRESENTED TO

Thameemunisha M

Guide, ISRO URSC

PRESENTED BY

Dhavan M

Intern

Code Design Report

Software design of the GNSS based orbit determination and trajectory prediction tool

GNSS Based Orbit Determination and Trajectory Prediction · ISRO URSC 2026

November 2026

Index

Code design report

1. Design goals
2. Technology stack
3. The code base at a glance
4. Architecture
5. Package design in detail
6. Data model
7. Dynamic behaviour
8. Concurrency and process model
9. Cut-off safety by design
10. Reproducibility, provenance and error handling
11. User interfaces
12. Build, packaging and deployment
13. Testing strategy
14. Design patterns used
15. Known design debt

Code design report

This report describes the software design of the GNSS-based orbit determination and trajectory prediction tool: its goals, technology, structure, the responsibilities of its packages and classes, the data it works on, how a run executes, how it handles concurrency, errors, cut-off safety and reproducibility, its user interfaces, how it is built and deployed, and how it is tested. It closes with the design patterns used and the known design debt.

Every name in this report (classes, methods, settings, endpoints) is taken from the code on the branch `claude/leo-trajectory-prediction-java-ihupky`.

---

1. Design goals

The design follows from seven requirements:

#	Requirement	How the design meets it
R1	**Run fully offline** (air-gapped machines)	single executable jar with every dependency shaded in; bundled gravity, ephemeris and antenna data; the GUI is a local web page with no internet resources; a private Java runtime in the installer
R2	**Cut-off safety**: never use information from after the cut-off	dedicated cut-off-aware providers for every time-varying input (section 9)
R3	**Physical fidelity** from a validated library	all physics on the run path delegated to Orekit 13.1.2 through a thin adapter package
R4	**Reproducibility**	every run stores its configuration, the SHA-256 hash of every input, its log, its result and its output (section 10)
R5	**Usable by non-programmers**	a guided web GUI with a data catalogue that recognises files by content, presets, explanations for every setting and help search
R6	**Scriptable**	the same run engine behind a command line (`run` executes any saved GUI configuration)
R7	**Minimal dependencies**	one runtime dependency (Orekit, which brings Hipparchus); JSON, HTTP server, archive readers and search written on the JDK alone

---

2. Technology stack

Layer	Technology
Language	Java 17 (`maven.compiler.release` 17)
Flight-dynamics library	Orekit 13.1.2 + Hipparchus 4.0.2 (Apache 2.0)
Build	Maven; `maven-shade-plugin` 3.5.3 produces one self-contained jar
Tests	JUnit Jupiter 5.10.2, `maven-surefire-plugin` 3.2.5
Web server	JDK built-in `com.sun.net.httpserver.HttpServer`
Front end	plain HTML, CSS and JavaScript modules; three.js (bundled) for the 3-D globe
Packaging	`jlink` (private runtime) + `jpackage` (native launcher), scripts for Linux/macOS and Windows

---

3. The code base at a glance

Measure	Value
Main source	192 Java files, 26 668 lines

Measure	Value
Tests	47 test files, 4 267 lines
Front end	15 JavaScript modules, one HTML page, one stylesheet, parameter registry `params.json`
Bundled data	EGM96 gravity, FES2004 ocean tides, DE440 ephemeris (installer), IGS GPS antenna file

Packages

Base package `org.ursc.trajectory`. The packages fall into three groups: the **run path** (everything a GUI or `run` execution touches), the **support** code shared by everyone, and the **legacy** in-house physics library kept for the older CLI research commands.

Package	Files	Group	Responsibility
`orekit`	11	run path	adapter to Orekit: data context, GPS products, dynamics, carrier-phase OD, forecast
`pipeline`	19	run path	run configuration and result, the OD + forecast pipeline, cut-off-safe inputs, comparisons
`gnss`, `gnss.products`	16	run path	RINEX readers, single-point positioning, GPS product tiers and eligibility
`data`	2	run path	data catalogue and folder watcher
`app.gui`	5	run path	local web server, run store, run report, window workers
`app`, `app.command`, `app.term`	23	interface	command-line entry point, subcommands, terminal output helpers
`io`	8	support	OEM, POEORB, TLE, archives (tar.gz, .Z), KML/CZML
`search`	8	support	offline help search (BM25) with optional answer providers
`util`, `time`, `math`	10	support	JSON, the program's own date type, vectors and small linear algebra
`frames`, `bodies`, `attitude`	20	support + legacy	EOP providers, DE440 reader, attitude files (used by the run path); own frames and bodies (legacy)
`forces`, `ode`, `orbits`, `propagation`, `estimation`, `config`, `analysis`	70	legacy	in-house force models, integrators, propagators and estimator

The dependency rule: **only the `orekit` package and three `pipeline` classes import Orekit** (`OdForecastPipeline`, `ObsLoader`, `PropagateFromState`). Everything else talks to the run engine through the program's own types, so the GUI, catalogue and file formats do not depend on Orekit's API.

---

4. Architecture

4.1 Layered view

```
┌──────────────────────────────────────────────────────────────────────┐
│ PRESENTATION      Browser front end (index.html + js/*.js, three.js) │
│                   ── HTTP/JSON + Server-Sent Events, loopback only ──│
│                   GuiServer · RunStore · RunReport · WindowWorker    │
│                   Cli · Command implementations (RunCommand, ...)    │
├──────────────────────────────────────────────────────────────────────┤
│ APPLICATION       Runs (validate, execute)                           │
│                   OdForecastPipeline · PropagateFromState            │
│                   DataCatalog · FolderWatcher · SearchService        │
├──────────────────────────────────────────────────────────────────────┤
│ DOMAIN / ADAPTER  orekit.*: OrekitSession, OrekitGpsProducts,        │
│                   OrekitDynamics, OrekitCarrierPhaseOd, OrekitForecast,│
│                   SparseGaussNewtonOptimizer, GpsAntennas,           │
│                   PhaseWindUpModifier, CpodBodyFrame                 │
│                   gnss.*: RinexObsReader, SppEngine, GnssProductResolver│
├──────────────────────────────────────────────────────────────────────┤
│ INFRASTRUCTURE    Environment (EOP, space weather, attitude, DE440)  │
│                   CutoffEop · ForecastSpaceWeatherProvider ·         │
│                   CutoffJb2008Inputs · io.* readers/writers · util.Json│
├──────────────────────────────────────────────────────────────────────┤
│ LIBRARY           Orekit 13.1.2 · Hipparchus 4.0.2 (shaded into the jar)│
└──────────────────────────────────────────────────────────────────────┘
```

Each layer calls only the layer below it. The run engine has a single entry point, `Runs.execute(RunConfig, ProgressListener) → RunResult`, used identically by the GUI, the `run` command and the window workers.

4.2 Component responsibilities in one sentence each

- **`GuiServer`**: serves the front end and a JSON API on 127.0.0.1, runs one job at a time, streams progress.
- **`Runs`**: validates a configuration and dispatches to the right workflow.
- **`OdForecastPipeline`**: the OD + forecast workflow from raw files to a trajectory.
- **`OrekitSession`**: one run's Orekit data context, frames and bodies.
- **`OrekitCarrierPhaseOd`**: the estimator (measurements, editing, solving).
- **`OrekitForecast`**: propagation, sampling, covariance, output frame.
- **`DataCatalog`**: knows every input file, its kind and time span, and suggests inputs for a cut-off.
- **`RunStore`**: keeps each run on disk so it can be reopened, compared and reproduced.

---

5. Package design in detail

5.1 `orekit` : the adapter to the physics library

Class	Lines	Responsibility	Key collaborators
`OrekitSession`	209	Builds a fresh `LazyLoadedDataContext` per run: writes the leap-second table, loads EOP from the cut-off-safe provider, EGM96 or an ICGEM file, FES2004 tides and DE440. Exposes `gcrf`, `eme2000`, `itrf`, `earth`, `sun`, `moon`, `utc`, `ut1`, `gravity(degree, order)`. `AutoCloseable`.	`EopProvider`, Orekit `DataContext`
`OrekitDates`	32	Converts dates and vectors between the program's own types and Orekit's	—
`OrekitGpsProducts`	431	Loads SP3, RINEX clock and RINEX navigation files with Orekit parsers and evaluates them with Orekit propagators; knows which products refer to the centre of mass	`GnssOrbitClockSource`, `ProductLoader`
`OrekitGpsSource`	51	One GPS product evaluated by Orekit, exposed as a `GnssOrbitClockSource` so the existing tier and eligibility logic applies unchanged	`GnssProductResolver`
`OrekitDynamics`	410	Builds the force-model list from the `RunConfig` switches; selects the atmosphere; builds the nominal or measured attitude; builds the propagator and the propagator builder for estimation; adapts the	`RunConfig.Forces`, `Environment`

Class	Lines	Responsibility	Key collaborators
		program's space-weather providers to Orekit's NRLMSISE-00 and JB2008 inputs	
`OrekitCarrierPhaseOd`	714	The orbit-determination estimator (section 7.2)	`BatchLSEstimator`, `SparseGaussNewtonOptimizer`
`OrekitForecast`	144	Propagates the estimate, samples the output, propagates the covariance with the state-transition matrix, converts to the output frame	`MatricesHarvester`
`SparseGaussNewtonOptimizer`	181	A Hipparchus `LeastSquaresOptimizer` that forms the normal equations from each Jacobian row's non-zero entries	`BatchLSEstimator`
`GpsAntennas`	100	Reads the IGS ANTEX file and builds ionosphere-free phase-centre patterns per GPS satellite	Orekit `AntexLoader`
`PhaseWindUpModifier`	83	Carrier-phase wind-up as an Orekit `EstimationModifier`	`OneWayGNSSPhase`
`CpodBodyFrame`	47	Wraps any Orekit attitude law in the Sentinel-3 body convention (180° about z)	`YawCompensation`, `NadirPointing`

5.2 `pipeline`: the run engine

Class	Lines	Responsibility
`RunConfig`	289	Every user setting in nested public-field sections with generic defaults; JSON round trip (section 6.1)
`RunResult`	128	Everything a run produced: status, provenance, diagnostics and states (section 6.2)
`Runs`	77	Front-end entry point: `validate(cfg)` returns problems, `execute(cfg, listener)` runs the chosen workflow
`OdForecastPipeline`	607	The OD + forecast workflow (section 7.1); includes `validate`, `gibbs`, input hashing

Class	Lines	Responsibility
`PropagateFromState`	184	Propagation workflow from a typed state, an OEM or a POEORB state
`Environment`	252	Builds the run's non-Orekit inputs: EOP, space weather (observed or forecast mode), JB2008 inputs, attitude files, DE440 and ANTEX locations; records every default it had to use
`ObsLoader`	278	Opens GNSSRD archives or RINEX files, thins them for slip detection, parses them with Orekit, merges epochs, reads the antenna offset
`CutoffEop`, `CutoffJb2008Inputs`	35, 56	Hold Earth orientation and JB2008 drivers at the cut-off value
`ReferenceComparison`	104	Interpolates POEORB to the run's epochs and gives 3-D and radial/along/cross differences (display and scoring only)
`RunCompare`	127	Difference between two runs' trajectories in the radial/transverse/normal frame
`ManeuverCheck`	83	Flags fits whose residual pattern suggests an unmodelled manoeuvre or bad data
`Passes`	104	Ground-station visibility passes from the trajectory
`FrameUtil`	222	State and covariance conversions between GCRF, ITRF and EME2000
`OemExport`	87	Builds the CCSDS OEM with provenance comments
`DemoDataset`	377	Writes a labelled simulated dataset in real file formats for trying the workflow
`ProgressListener`	23	Progress, log and cancellation callbacks (section 8)
`InputFiles`, `TimeUtil`	39, 48	Expand file/folder lists; ISO-8601 UTC parsing

5.3 `gnss` and `gnss.products`: GNSS data

Class	Responsibility
`RinexObsReader`	RINEX 3.0x observation reader keeping loss-of-lock flags and the antenna-offset header
`RinexNavReader`, `GpsBroadcastEphemeris`	RINEX 3/4 GPS navigation messages and the IS-GPS-200 orbit algorithm
`SppSolver`, `SppEngine`, `SppSolution`, `SatelliteRange`	single-point positioning used to start the orbit

Class	Responsibility
`GnssOrbitClockSource`	the interface every GPS product implements (section 5.10)
`GnssProductResolver`	per-epoch choice of product: eligibility at the cut-off first, then tier priority Final > Rapid > Ultra-rapid > Broadcast
`ProductLoader`, `IgsProductName`	detect product kind from content; parse IGS long and short file names (issue time, latency)
`Sp3Reader`, `RinexClkReader`, `PreciseSource`, `BroadcastSource`	in-house product readers (used by SPP and the legacy tools; the OD uses the Orekit parsers)

5.4 `data`: the data catalogue

`DataCatalog` (580 lines) keeps a persistent index of every file the user has added: single files, many files, folders or many folders. Each file is **identified from its content**, not its name, into one of eleven kinds (`Kind` enum: `GNSS_OBSERVATIONS`, `GPS_BROADCAST_NAV`, `IGS_ORBIT_SP3`, `IGS_CLOCK`, `POEORB`, `ATTITUDE_PROQUA`, `EOP`, `SPACE_WEATHER`, `DE_EPHEMERIS`, `GRAVITY_FIELD`, `OEM`, plus `UNKNOWN`). Each kind maps to the `RunConfig` input it feeds. For each file the catalogue records its time span, satellite and size. `suggest(cutoff, arcHours, spanDays, …)` returns the files a run needs for a given cut-off. `FolderWatcher` (150 lines) watches folders recursively and indexes new files once their size stops changing, so files still being copied are not read half-written.

5.5 `app.gui`: the GUI back end

Class	Responsibility
`GuiServer` (1 442 lines)	HTTP server bound to the loopback interface; static files and a JSON API (section 11); a single-thread job executor; Server-Sent Events for progress; window and sweep runs
`RunStore`	run history on disk, unique run identifiers
`RunReport`	self-contained printable HTML report of one run
`TrajectoryView`	visualisation payload: Earth-fixed and inertial positions, ground track, Sun direction
`WindowWorker`	`main` of the child JVM that runs one window day and saves it exactly like an interactive run

5.6 `app` and `app.command`: the command line

`Cli` dispatches to a `Command` implementation by name. `gui` and `run` use the Orekit engine; the others (`propagate`, `predict`, `forecast`, `fit`, `validate`, `backtest`, `compare`, `decay`, `tle`, `spp`, `demo-data`) are research tools, and those built on the in-house physics library are labelled "legacy" in the help. `Args` is a dependency-free argument parser.

5.7 `io` : file formats

Class	Format
`Oem`	CCSDS OEM 2.0 KVN, read and write, with covariance
`PoeOrbReader`	Sentinel POEORB precise orbit (`.EOF`)
`TLE`	two-line elements
`TarGz` , `UnixCompress` , `ProductInput`	`.tgz` archives (GNSSRD), Unix `.Z` (legacy IGS), transparent gzip
`TrackExport`	KML and CZML ground tracks
`CsvEphemerisWriter`	CSV output for the legacy propagators

5.8 `search` : offline help

`SearchService` indexes the help pages, the parameter registry and the project documents into `Passage` s and ranks them with `Bm25Index` (Okapi BM25). Answers come from a chain of `AnswerProvider` s tried in order: `OfflineExtractiveProvider` (always available, quotes and cites the best sentences), `OllamaProvider` (a local model, if installed) and `CloudProvider` (only if the user configures a key; never used offline). `AiConfig` stores the provider choice and never holds secrets.

5.9 `util` , `time` , `math`

`Json` (393 lines) parses and writes JSON with no dependency and maps public-field objects such as `RunConfig` to and from JSON. `AbsoluteDate` is the program's own instant type (SI seconds from J2000 TAI), used everywhere outside the `orekit` package; `OrekitDates` converts at the boundary. `Vector3D` , `RotationMatrix` and `LinearSolver` serve the SPP and legacy code.

5.10 Key interfaces and extension points

Interface	Methods	Implementations	Extend to ...
`GnssOrbitClockSource`	`productClass()` , `issueTime()` , `covers(prn, t)` , `positionEcef(prn, t)` , `clockBias(prn, t)` , `availableBy(cutoff)` , `isPredicted(t)`	`OrekitGpsSource` , `PreciseSource` , `BroadcastSource`	add a GNSS constellation or product type
`ProgressListener`	`stage(name, fraction)` , `log(msg)` , `cancelled()`	GUI job runner, CLI, `NONE`	add a new front end

Interface	Methods	Implementations	Extend to ...
`EopProvider`	UT1-UTC, polar motion	`IersEopProvider`, `Finals2000AEopProvider`, `ConstantEop`, `CutoffEop`	add an EOP source
`SpaceWeatherProvider`	daily F10.7, 81-day F10.7, Ap	observed CSSI provider, `ForecastSpaceWeatherProvider`, constant	add a space-weather forecast
`Command`	`name()`, `description()`, `run(Args)`	15 commands	add a CLI command
`AnswerProvider`	`name()`, `available()`, `status()`, `answer(question, passages)`	offline, Ollama, cloud	add an answer model
Orekit `EstimationModifier`	`modify(...)`	`PhaseWindUpModifier` (plus Orekit's own)	add a measurement correction

---

6. Data model

6.1 `RunConfig` : the single source of truth for a run

A run is fully described by one `RunConfig`, serialised as JSON. The GUI edits it, the run store saves it, the `run` command reads it, and window workers receive it as a file.

Section	Main fields	Notes
top level	`runName`, `workflow` (`OD_FORECAST` / `PROPAGATE`), `cutoffUtc`	cut-off is mandatory for OD
`inputs`	`gnssrd[]`, `gpsProducts[]`, `poeorb[]`, `proqua[]`, `eopFile`, `spaceWeatherFile`, `de440File`, `antexFile`, `gravityFile`	lists accept files or folders
`products`	`mode` (forecast / reconstruction), `eligibility`, `tier` (`AUTO`, `FINAL`, `RAPID`, `ULTRA_RAPID`, `BROADCAST`)	forcing a tier is for comparison runs
`od`	`arcHours`, `phaseRateSeconds`, `codeSigma`, `phaseSigma`, `estimateCd`, `estimateCr`, `antennaOffset`, `gpsAntennaPhaseCentre`, `phaseWindUp`, `maxIterations`, SPP gates (`samplingSeconds`, `maxPdop`, `maxSppRms`, `pseudorangeSigma`, `minSatellites`)	

Section	Main fields	Notes
`forecast`	`spanDays`, `outputStepSeconds`, `frame` (`EME2000`, `GCRF`, `ITRF`), `covariance`, `covarianceEveryNSteps`	
`spacecraft`	`objectName`, `objectId`, `massKg`, `shape` (`CANNONBALL` / `BOX_WING`), areas, `cd`, `cr`, `comBodyX/Y/Z`, `attitude` (`NOMINAL_YAW_STEERING` / `MEASURED_PROQUA`), box-wing faces	
`forces`	`gravity` (degree, order), `drag` + `dragModel` (`NRLMSISE00`, `JB2008`, `HARRIS_PRIESTER`, `EXPONENTIAL`), `srp`, `albedo`, `solidTides`, `oceanTides`, `relativity`, third bodies, empirical	each a `Toggle`
`weather`	constant values, after-cut-off policy, `forecastAp`	
`eop`	after-cut-off policy (`PERSIST` / `USE_FILE`)	
`integrator`	`positionToleranceM`, `maxStepSeconds`	
`initialState`	source (`TYPED`, `OEM_FILE`, `POEORB_FILE`) and values	propagation workflow
`window`	`startDate`, `endDate`, `cutoffTimeUtc`, `stepDays`, `parallelRuns`	window runs

Defaults live in the class (generic) and in GUI profiles (`profiles/showcase.json`, the tested Sentinel-3A settings). `params.json` documents every field for the GUI and the help search; a test (`ParamsRegistryTest`) keeps it in step with the class.

6.2 `RunResult`

Group	Fields
Outcome	`status` (`SUCCESS`, `FAILED_VALIDATION`, `FAILED_SPP`, `FAILED_OD`, `CANCELLED`, `ERROR`), `errors`, `warnings`, `flags`, `defaultsUsed`, `log`, `elapsedSeconds`
Identity	`workflow`, `runName`, `label`, `objectName`, `objectId`, `cutoffUtc`
Provenance	`inputHashes` (file → SHA-256), `productUsage`, `productRejected`, `productFallbacks`
Diagnostics	`spp` rows (per epoch: product, satellites, PDOP, RMS, accepted, reason), `od` (convergence, iterations, observations, pre/post-fit RMS, variance factor, code and phase RMS, arcs, slips, estimated Cd/Cr, covariance)
Output	`states` (epoch, position, velocity, optional covariance, in GCRF and the output frame)

6.3 Persisted run

`RunStore` writes each run to `runs/<id>/`:

File	Content
`config.json`	the exact `RunConfig`
`result.json`	the `RunResult` without the states
`trajectory.json`	the states and visualisation payload
`*.oem`	the CCSDS OEM product
`log.txt`	the run log

---

7. Dynamic behaviour

7.1 One OD + forecast run

`OdForecastPipeline.run(cfg, listener)`:

#	Step	Class / method	Progress stage
1	Validate the three mandatory inputs and parse the cut-off	`validate`	—
2	Expand file lists, hash every input (SHA-256)	`InputFiles.expand` , `hash`	—
3	Refuse mixed spacecraft	`ObsLoader.spacecraft`	—
4	Build the environment (EOP, space weather, attitude)	`Environment.setUp`	—
5	Open the Orekit session for [arc start − 1 day, forecast end + 1 day]	`openSession` → `OrekitSession.open`	—
6	Read observations (thinned to 10 s)	`ObsLoader.loadWithOrekit`	2 %
7	Load GPS products, drop those not published by the cut-off	`OrekitGpsProducts.load`	6 %
8	Single-point positions every 300 s	`SppEngine`	
9	Gibbs initial orbit, then position fit	`gibbs` , Orekit `BatchLSEstimator`	
10	Carrier-phase OD; on failure with Cd/Cr, refit with them fixed and warn	`OrekitCarrierPhaseOd.estimate`	
11	Forecast, covariance, output frame	`OrekitForecast`	
12	Manoeuvre check, product usage summary	`ManeuverCheck`	

#	Step	Class / method	Progress stage
13	Close session and environment (try-with-resources); record elapsed time	—	100 %

Any exception becomes `status = ERROR` with the message in `errors`; the method never throws.

7.2 Inside the estimator

`OrekitCarrierPhaseOd.estimate()` runs an outer editing loop around Orekit's estimator:

```
build arcs (LLI, gaps, MW, GF, product change)
measurements(clocks, ambiguities)            one ObservableSatellite per epoch,
                                             one AmbiguityDriver per arc
(ArcAmbiguityCache)
repeat
    solve(measurements)                      BatchLSEstimator +
SparseGaussNewtonOptimizer
    n1 = splitAtResidualSteps()              split arcs at residual jumps
    n2 = rejectOutliers()                    drop residuals > 4 sigma
    snapshot(clocks, ambiguities)            carry values into the rebuilt problem
until n1 + n2 == 0
re-weight code and phase by their post-fit RMS, final solve
return Result (state, covariance, RMS, variance factor, arcs, slips, Cd/Cr)
```

7.3 Window run

1. `POST /api/window` with a base configuration and a date range.
2. `GuiServer` computes the cut-offs and the worker count `min(cores − 1, (memory GB − 2) / 3)`, at least 1, unless `window.parallelRuns` is set.
3. A fixed thread pool starts one **child JVM per day** (`runInWorker`): the configuration is written to a temporary file and `WindowWorker` is launched with `-Xmx3g`; its output goes to `window-workers.log`.
4. Each worker runs `Runs.execute` and saves the run in the shared `RunStore`.
5. As each day finishes, `windowDayDone` records its row (kept in date order) and pushes an event to the browser, which updates the trend charts live.

7.4 Interactive run from the GUI

`POST /api/run` → if a job is already running, HTTP 409 → otherwise the job is submitted to the single-thread executor → the job's `ProgressListener` forwards stages and log lines to the SSE stream → on completion the run is saved and the browser loads its result. `POST /api/runs/<id>/cancel` sets the flag that `cancelled()` returns.

---

8. Concurrency and process model

Thread / process	Purpose
`ursc-http` (cached pool)	serves HTTP requests; never runs physics
`ursc-run` (single thread)	executes the one interactive run, sweep or window coordinator at a time
`ursc-sse` (scheduled)	pushes progress events and heartbeats
`FolderWatcher` thread	watches catalogue folders
window pool + child JVMs	one process per window day

Two design decisions follow from Orekit's data context being **process-wide** (`DataContext.setDefault`):

- inside one JVM, only one run may hold an Orekit session at a time, hence the single-thread job executor and the HTTP 409 answer when busy;
- parallelism is achieved with **processes**, not threads, so each day has its own data context and cut-off-safe EOP, with no shared mutable state.

---

9. Cut-off safety by design

Every time-varying input has a dedicated guard. The guards are in the infrastructure layer, so the physics code cannot accidentally bypass them.

Input	Guard	Behaviour after the cut-off
GPS products	`GnssOrbitClockSource.availableBy(cutoff)`, `GnssProductResolver`	a product is used only if published by the cut-off (nominal latency by tier); broadcast messages only with reference time ≤ cut-off
Observations	`ObsLoader` window	read only up to the cut-off
Earth orientation	`CutoffEop` → `OrekitSession`	held at the cut-off values
Leap seconds	written by `OrekitSession` from the program's table	—
Space weather (MSIS)	`ForecastSpaceWeatherProvider`	F10.7 persisted, Ap = `weather.forecastAp`
Space weather (JB2008)	`CutoffJb2008Inputs`	held at the cut-off values
Measured attitude	`CompositeAttitude`	quaternions only up to the cut-off, then the nominal law

Input	Guard	Behaviour after the cut-off
POEORB	`ReferenceComparison`	read only for display and scoring, never by the run

---

10. Reproducibility, provenance and error handling

- **Provenance:** SHA-256 of every input, the product used for every epoch and every fallback, and every default the environment had to assume (`defaultsUsed`).
- **Reproduction:** the stored `config.json` re-runs the identical run from the GUI or with `run config.json` .
- **Status, not exceptions:** validation problems, SPP failure, OD failure, cancellation and unexpected errors each have their own `Status` . Warnings (for example "refitted with Cd fixed") and flags (for example a suspected manoeuvre) travel with the result and appear in the GUI and in `RunReport` .
- **Graceful degradation:** missing optional inputs fall back to documented defaults (constant space weather, zero EOP, bundled gravity) and are listed in `defaultsUsed` .

---

11. User interfaces

11.1 Web API

The server binds to `127.0.0.1` only. A random token, passed to the browser in the URL fragment (which browsers never send to servers), is required on API calls, so other local programs and web pages cannot drive it.

Endpoint	Method	Purpose
`/api/status`	GET	workspace, the running job (id, stage, fraction), search index state
`/api/events`	GET (SSE)	progress, log lines, window-day completions
`/api/fs`	GET	browse local folders for adding data
`/api/config`	GET	default (showcase) configuration
`/api/catalog`	GET	catalogue contents
`/api/catalog/add` , `/remove` , `/watch` , `/unwatch` , `/rescan`	POST	manage catalogue entries and watched folders
`/api/catalog/suggest`	GET	inputs for a cut-off
`/api/catalog/cutoff`	GET	latest cut-off the data supports
`/api/run`	POST	start an interactive run

Endpoint	Method	Purpose
`/api/demo`	POST	generate the simulated demo dataset
`/api/runs`	GET	run history
`/api/runs/<id>`	GET / DELETE	one run's result
`/api/runs/<id>/trajectory`, `/reference`, `/passes`	GET	visualisation, POEORB comparison, ground passes
`/api/runs/<id>/oem`, `/log`, `/kml`, `/czml`, `/report`	GET	downloads
`/api/runs/<id>/cancel`	POST	cancel
`/api/presets`	GET / POST / DELETE	saved configurations
`/api/search`	GET	help search and answer
`/api/compare`	GET	compare several runs
`/api/stations`	GET / POST	ground stations for pass prediction
`/api/sweep`	POST	run one setting at 1–12 values
`/api/window`	POST	start a window run
`/api/windows`	GET / DELETE	window-run summaries
`/api/shutdown`	POST	stop the server

11.2 Front end

Module	Role
`app.js`, `ui.js`, `api.js`	application shell, shared widgets, API client with token
`data.js`	data tab: add files and folders, catalogue table
`run.js`	run tab: form generated from `params.json`, presets, cut-off from data
`results.js`, `charts.js`	results tab: diagnostics, residuals, error charts
`globe.js`, `map2d.js`, `spacecraft.js`	3-D globe (three.js), 2-D map, Sentinel-3A and generic models
`windowview.js`	window-run trends, live
`compare.js`, `history.js`	run comparison and history
`help.js`, `presentation.js`	help search and the built-in presentation

All third-party code and map data (`vendor/three.module.min.js` , `borders-50m.json`) are bundled with their licences, so the page loads without internet.

11.3 Command line

`java -jar leo-trajectory-0.1.0.jar <command>` ; `gui` starts the server and opens the browser, `run <config.json>` executes a saved configuration headless, the rest are research tools.

---

12. Build, packaging and deployment

1. `mvn package` compiles, runs the tests and shades Orekit and Hipparchus into `target/leo-trajectory-0.1.0.jar` .
2. `packaging/build-bundle.sh` (or `.ps1` on Windows) copies the jar, DE440 and the ANTEX file, uses `jlink` to build a private Java runtime with only the needed modules, and `jpackage` to build a native application folder with a launcher.
3. The result runs on a machine with nothing installed and no network (`packaging/README-OFFLINE.md`).

Licensed data that cannot be redistributed (NRLMSIS 2.0, DTM2020) is deliberately not part of the repository or the bundle.

---

13. Testing strategy

Level	Tests	What they establish
Estimator closed loop	`OrekitCarrierPhaseOdTest` with `testutil/SyntheticGnss`	simulated code + phase with clock, ambiguities, slips and antenna offset: orbit recovered to 4.9 mm, slips found, variance factor ≈ 1
Pipeline end to end	`PipelineTest` , `DemoDatasetTest` , `PropagateFromStateTest`	full run from files: 4 mm at the cut-off, 7 mm at +1 day on synthetic data
File formats	`RinexObsReaderTest` , `RinexNavReaderTest` , `Rinex4NavReaderTest` , `PoeOrbReaderTest` , `TarGzTest` , `UnixCompressTest` , `GnssProductsTest`	readers against real-format samples
GNSS	`SppSolverTest` , `SppEngineTest` , `GpsBroadcastEphemerisTest` , `BroadcastMergeTest`	positioning and broadcast orbits
Frames and time	`FramesTest` , `FrameUtilTest` , `TimeScaleTest` , `Finals2000AEopProviderTest`	transformations and time scales
Application	`DataCatalogTest` , `ParamsRegistryTest` , `RunCompareTest` , `PassesTest` ,	catalogue identification, registry in step with `RunConfig` ,

Level	Tests	What they establish
	`CliSupportTest`, `PredictCommandTest`, `SppCommandTest`	comparisons
Legacy physics	`ForceModelsTest`, `Nrlmsise00Test`, `Jb2008Test`, `J2PerturbationTest`, `DsstPropagatorTest`, `BatchLeastSquaresTest`, …	the in-house library used by the legacy commands

All 123 tests pass. Beyond unit tests, the 61-day Sentinel-3A campaigns act as system tests against POEORB.

---

14. Design patterns used

Pattern	Where
Adapter	`orekit` package (program types ↔ Orekit), `OrekitGpsSource` (Orekit product → `GnssOrbitClockSource`), `MsisAdapter` / `Jb2008Adapter`
Facade	`Runs`, `OdForecastPipeline`, `OrekitSession`
Strategy	atmosphere model, attitude source, answer providers, EOP and space-weather providers
Decorator	`CpodBodyFrame` around an attitude law; `CutoffEop` and `ForecastSpaceWeatherProvider` around observed providers
Chain of responsibility	product tiers in `GnssProductResolver`; answer providers in `SearchService`
Command	CLI subcommands
Observer	`ProgressListener`, Server-Sent Events, `FolderWatcher`
Resource management	`OrekitSession` and `Environment` as `AutoCloseable` in try-with-resources
Data-transfer objects	`RunConfig`, `RunResult` (public fields, JSON-mapped)
Process isolation	`WindowWorker` child JVMs

---

15. Known design debt

Item	Effect	Remedy
Legacy in-house physics library (≈ 70 files) beside the Orekit engine	two code bases; easy to confuse	move to a separate module or archive
`GuiServer` is 1 442 lines	hard to navigate	split into handlers per API area

Item	Effect	Remedy
Per-epoch `ObservableSatellite` workaround	larger problem than necessary	return to Orekit time-span drivers once the defect is fixed upstream
In-house cycle-slip tests, OEM writer, POEORB reader	duplicate Orekit features	migrate where Orekit's versions meet the cut-off rules
Daily Ap repeated for NRLMSISE-00's 3-hourly history	weaker storm response	pass 3-hourly Ap
One interactive run per JVM	GUI is busy during a run	run interactive jobs in a worker process too